Spatio-Temporal Railway Network Analytics Using Machine Learning, Graph Analytics and GIS

This is proposed as an intelligent railway-network analytics system rather than a simple statistics project. The system will combine historical Indian Railways statistics with station and rail-line geospatial data. The cleaned data will be used to study how the network has changed over time, represent the railway system as a graph, identify structurally important stations and corridors, group stations with similar network characteristics, and conduct a controlled link-prediction experiment. The final output will be presented through an interactive Streamlit dashboard.

The project is deliberately built around several CSE and Data Science competencies, but each component has a clear purpose. It includes data acquisition and cleaning, geospatial processing, graph construction, algorithmic network analysis, unsupervised learning, supervised classification, model evaluation and application development. The project therefore produces both an analytical study and a working software demonstrator.

Core module	What it contributes
Module 1 – Infrastructure analytics	Historical analysis of gauge, route length, electrification and selected railway assets.
Module 2 – Railway graph intelligence	Build a station-level graph and calculate centrality, communities and vulnerability measures.
Module 3 – Machine learning	Cluster stations with PCA + K-Means and perform graph link prediction with classification models.
Module 4 – Web application	Present maps, network insights, model results and station-level information through Streamlit.

2. Problem Statement

Indian Railways produces a large amount of statistical information, while open geospatial platforms contain station and rail-line geometry. These sources are useful individually, but they are not normally combined into a student-friendly analytical system. A conventional project may stop after plotting route kilometres, passenger numbers or electrification. That is useful EDA, but it does not fully demonstrate the skills expected from a final-year CSE Data Science project.

The proposed work addresses this gap by creating a unified railway analytics pipeline. Historical statistics are used to understand long-term infrastructure change, while station and rail-line data are converted into a graph. The graph is then analysed using network measures, and machine-learning models are applied to station clustering and link prediction. The purpose is not to replace railway planning practice, but to demonstrate how data-science and graph methods can extract meaningful structure from a national transport network.

This project should be presented as an analytics and modelling system. It should not claim that the ML

model decides where Indian Railways must construct new lines. Link prediction is treated as a controlled graph-learning experiment in which known links are hidden and the model is tested on whether it can recover them.

3. Aim, Objectives and Scope

Aim

To design and implement a data-driven railway network analytics platform for India by integrating historical railway statistics, geospatial data, graph algorithms and machine-learning models.

Specific objectives

1. Create a clean, documented dataset that combines railway statistics, gauge history, electrification information, station attributes and rail-network geometry.

2. Study long-term changes in broad gauge, metre gauge and narrow gauge route kilometres and selected infrastructure indicators.

3. Construct a station-level railway graph and calculate measures such as degree, betweenness, closeness and PageRank.

4. Identify communities or regional structure in the railway graph and examine network robustness through controlled node-removal experiments.

5. Create numerical station features and use PCA and K-Means to identify groups of stations with similar network characteristics.

6. Develop a link-prediction dataset from the graph and compare Logistic Regression and Random Forest using appropriate classification and ranking metrics.

7. Build a Streamlit dashboard that allows a user to inspect historical trends, station/network metrics, maps and model outputs.

The project does not need to reproduce every railway line, train and timetable in India at perfect operational accuracy. The goal is to build a reproducible analytical pipeline. If the full national rail graph becomes too large for the available computer, the implementation can use a state, railway zone, or a carefully filtered national station network while retaining the same methodology. The report should clearly state the geographic and temporal scope actually used.

CSE / Data Science area	Where it appears in RailNet-India
Data engineering	Combining CSV, government statistics and GIS files; cleaning names, codes, missing values and data types.
Algorithms & data structures	Representing stations and rail connections using graphs; shortest paths, components and centrality.
Database / storage	Storing cleaned tables and model-ready data in CSV/Parquet or SQLite.

CSE / Data Science area	Where it appears in RailNet-India
Data Science	EDA, feature engineering, scaling, dimensionality reduction and clustering.
Machine learning	Supervised link prediction using engineered graph features and model evaluation.
Geospatial computing	Coordinate systems, spatial joins and mapping with GeoPandas/Folium or Plotly.
Software engineering	Modular Python code, reusable preprocessing functions and a Streamlit front end.
Evaluation	Train/test separation, negative sampling, cross-validation where appropriate and metrics beyond accuracy.

5. Data Sources and Dataset Plan

Use official railway statistics for national totals and use open geospatial sources for station and line geometry. Every processed table should keep a source field and a data-vintage field. This is particularly important because the sources may refer to different years.

Dataset	Source	Main fields	Use in project
Historical gauge dataset	Existing project CSV: gauge_history_1947_1971_2025.csv	Year, BG route km, MG route km, NG route km, total route km	Time-series EDA and gauge-conversion analysis
National railway indicators	Indian Railways Year Book / existing key-statistics CSV	Route km, track km, electrification, locomotives, wagons, selected traffic indicators	Current / recent infrastructure snapshot
Zone electrification snapshot	Existing zone_electrification_2021.csv	Zone/railway, total BG route km, electrified BG route km, balance	Historical zone comparison only
Station master data	Government OGD station data and/or DataMeet station files	Station name, code, state, zone, latitude, longitude	Station-level joins and labels

Dataset	Source	Main fields	Use in project
Rail-line geometry	OpenStreetMap / Geofabrik India or regional extract	railway geometry, station/track tags, gauge/electrification where available	GIS map and graph construction
Optional timetable/route data	Government OGD timetable resource or another documented source	Train-stop sequence, distance, arrival/departure where available	Optional connectivity enrichment; not required for MVP

Minimum cleaned tables to create

Table	Recommended fields
gauge_history	year, bg_km, mg_km, ng_km, total_route_km, bg_share_pct, source
railway_key_stats	fiscal_year, metric, unit, value, source
stations	station_id, station_code, station_name, state, zone, latitude, longitude, source, data_vintage
rail_edges	source_station_id, target_station_id, distance_km(optional), line/gauge/electrification fields if available
station_features	station_id + graph metrics + GIS / infrastructure features used by PCA and clustering
link_prediction_samples	node_u, node_v, graph features, optional geographic features, label

6. Proposed System Architecture

The architecture below separates raw data from processing and modelling. This makes the project easier to debug and easier to explain during viva.

1. Data sources	Indian Railways statistics | OGD station data | OpenStreetMap / GIS
↓	
2. Data engineering	Cleaning, field standardisation, missing-value handling, coordinate checks, source/vintage

	tagging
↓	
3. Analytical layer	Historical EDA | GIS analysis | Graph construction | Graph features
↓	
4. Modelling layer	PCA + K-Means | Link prediction with Logistic Regression / Random Forest | model evaluation
↓	
5. Application layer	Streamlit dashboard | maps | station search | model results | exported tables/plots

7. Detailed Methodology

Do not begin with machine learning before the graph and station tables are stable. Each phase produces an output that becomes the input to the next phase.

Phase 1 – Define scope and obtain the raw data

Start by fixing the geographic scope (national, selected zones, or selected states) and the years used for historical analysis. Download or retain a copy of every raw file in a read-only raw_data folder. Record the source URL, download date, data-vintage date and any licensing/attribution requirement in a small metadata CSV.

- Use the existing gauge-history and key-statistics files for the historical module.
- Obtain station coordinates/codes from a documented station source.
- Obtain railway line/station geometry from OpenStreetMap/Geofabrik for GIS and graph construction.
- Keep raw files unchanged. All cleaning should write new files to processed_data.

Phase 2 – Data cleaning and standardisation

Use Pandas for tabular cleaning and GeoPandas for geographic files. Standardise column names to lowercase snake_case, convert numeric fields to proper numeric types, remove duplicate records, and standardise station codes. For geospatial data, check that latitude and longitude values are valid and work in a common coordinate reference system, normally WGS84 (EPSG:4326) for storage and web mapping.

- Check missing station codes, names and coordinates.
- Remove exact duplicates but do not remove legitimate stations only because names are similar.
- Create a stable station_id for joins.
- Normalize state and zone labels.
- Preserve unknown gauge/electrification values as unknown; do not automatically convert them to 'No'.

- Create a data_quality_notes column for records that need attention.

Phase 3 – Historical and descriptive analysis

Use the existing gauge history as the first analytical module. Calculate the share of BG, MG and NG in total route kilometres, yearly changes, long-period changes and selected growth/decline rates. Plot these against time. For recent infrastructure indicators, compare values only when the units and fiscal years match.

- Line plots: BG, MG, NG route km by year.
- Percentage-share plots: gauge share of the total network.
- Change features: year-on-year or interval change in route km.
- Selected electrification / rolling-stock comparisons.
- Write observations from the plots before attempting ML.

Phase 4 – Build the geospatial railway layer

Load station points and rail-line geometry in GeoPandas. Inspect the coverage visually and check for obviously misplaced coordinates. If necessary, clip the national file to the selected study area. For analysis involving distances, project the data to an appropriate metric CRS before measuring distances; convert back to EPSG:4326 for web display.

- Map station points and railway lines.
- Spatially join stations to state/district boundaries if those layers are added.
- Create basic spatial features such as nearest-neighbour distance or local station density if useful.
- Store the processed station and line layers as GeoPackage/GeoJSON/Parquet depending on tool support.

Phase 5 – Convert railway data into a graph

Use NetworkX to construct the graph. A node represents a station. An edge represents a direct connection in the processed railway network. If the data contains reliable distance information, store it as an edge weight. If the graph is constructed from line geometry, document exactly how stations were matched to railway segments and how consecutive connections were inferred.

- Node attributes: station code, name, state, zone, coordinates.
- Edge attributes: distance (if available), gauge/electrification tags if reliable, source.
- Start with an undirected graph unless the dataset clearly represents direction.
- Remove self-loops and exact duplicate edges.
- Inspect graph size, number of components and isolated nodes before calculating centrality.

Phase 6 – Graph analytics

Calculate network measures that have an intuitive railway interpretation. Degree centrality highlights stations with many direct graph connections. Betweenness centrality highlights nodes that sit on many shortest paths. Closeness indicates how efficiently a node can reach others within its connected component. PageRank provides another measure of structural importance. Community detection can be added to identify dense network groups.

- Degree / degree centrality – direct connectivity.
- Betweenness centrality – structural bridging role.
- Closeness centrality – reachability within a connected graph/component.
- PageRank – iterative importance score.

- Connected components – basic network integrity.
- Community detection – optional but useful for a stronger graph-analysis section.

Phase 7 – Network vulnerability experiment

To show an algorithmic experiment beyond descriptive centrality, remove nodes in a controlled order and measure how the graph changes. Compare targeted removal of high-centrality stations with random removal. The purpose is to study graph robustness, not to make operational claims about real railway disruption.

- Choose a baseline graph metric: largest connected component size, number of components, or global efficiency.
- Remove the top-k nodes by degree or betweenness.
- Repeat random removal several times and average the result.
- Plot the degradation curve and discuss which strategy damages connectivity faster.

Phase 8 – Create station-level machine-learning features

Combine the graph measures into a station_features table. This table is the input for station clustering. Use only meaningful and well-defined numeric features; do not include station name or station code as model features.

Feature	Meaning
degree	Number of direct graph neighbours
betweenness	How often a station lies on shortest paths
closeness	Relative reachability inside the graph/component
pagerank	Graph importance score
component_size	Size of the connected component containing the station
local_density / neighbour statistics	Optional structural features
latitude / longitude	Optional; use cautiously because geographic position can dominate clustering
spatial density	Optional station density in a defined radius

Phase 9 – PCA and K-Means station clustering

Scale numeric features using StandardScaler because graph measures have very different ranges. Use PCA primarily for dimensionality reduction and visualisation. Select a small number of principal components that retain a useful amount of variance. Run K-Means on the scaled features or on a suitable PCA representation. Evaluate multiple k values rather than choosing a cluster count arbitrarily.

- StandardScaler → puts features on comparable scales.

- PCA → reduces correlated features and enables 2D visualisation.
- K-Means → groups stations with similar numerical network profiles.
- Silhouette score → helps compare candidate cluster counts.
- Interpret clusters after training using their original feature averages; do not name them 'major hub' or 'local station' without checking the data.

Phase 10 – Link-prediction dataset construction

Link prediction provides the main supervised ML task. Use existing graph edges as positive examples. Hide a subset of known edges for testing. Generate non-edge node pairs as negative examples. The train/test procedure must avoid information leakage: graph features for a held-out edge should be computed from a graph in which that test edge is absent.

- Positive sample: a known direct graph edge.
- Negative sample: a selected node pair with no edge in the graph.
- Split edges into train/test sets before generating leakage-sensitive features.
- Use reproducible random seeds.
- If possible, sample negatives within a realistic geographic or graph-distance range so the task is not made artificially easy.

Phase 11 – Link-prediction features

Feature	What it measures	Why use it
Common neighbours	Count of neighbours shared by two stations	Simple topology similarity
Jaccard coefficient	Shared-neighbour overlap divided by union	Neighbourhood similarity
Adamic–Adar	Weights rare shared neighbours more strongly	Classic link-prediction score
Preferential attachment	Product of endpoint degrees	Tests hub-to-hub tendency
Shortest-path / graph distance	Optional and must be computed carefully	Structural separation
Geographic distance	Optional if coordinates are available	Adds spatial information

Phase 12 – Train classification models

Use Logistic Regression as a simple and interpretable baseline and Random Forest as a non-linear ensemble model. This is enough for a strong B.Tech comparison. More models should be added only if they answer a clear question. Train on the training set, tune a small number of hyperparameters, and evaluate on the untouched test set.

Phase 13 – Build the Streamlit application

The application is the final integration layer. It should read cleaned outputs and saved model results rather than performing every expensive graph computation from scratch on each page load. This makes the demo faster and more reliable.

8. Machine-Learning and Graph-Analysis Methods

Method	Role	Where used	How to evaluate / interpret
PCA	Dimensionality reduction / visualisation	Use on scaled station numerical features before visual cluster inspection.	Explained variance ratio and component loadings.
K-Means	Unsupervised clustering	Group stations by graph/spatial feature profile.	Silhouette score, cluster size and feature-profile interpretation.
Logistic Regression	Baseline supervised classifier	Predict held-out edge vs non-edge from link features.	Precision, recall, F1, ROC-AUC; coefficients can aid interpretation.
Random Forest	Non-linear supervised classifier	Compare with Logistic Regression on the same link-prediction dataset.	F1, ROC-AUC, PR-AUC; feature importance with caution.
Centrality measures	Graph algorithms	Identify structurally important stations.	Rankings and sensitivity to graph definition.
Community detection	Graph structure	Find groups of densely interconnected nodes.	Community sizes and map interpretation.
Node-removal robustness	Graph experiment	Study how network connectivity changes under targeted vs random removal.	Largest-component ratio / efficiency degradation curve.

9. What to Use Where: Tool-to-Task Mapping

Tool / library	Use it here	Purpose
Python	Entire project	Main programming language.
Pandas	CSV / statistical data cleaning	Gauge history, key statistics, station metadata, feature tables.
NumPy	Numerical operations	Array operations, feature calculations and support for modelling.
GeoPandas	GIS processing	Read spatial files, CRS conversion, spatial joins, station/rail-line processing.
Pyogrio / GeoPackage / GeoJSON	Spatial I/O	Efficiently read/write processed geographic layers.
NetworkX	Graph construction and analytics	Nodes/edges, centrality, components, shortest paths, PageRank and robustness experiments.
Scikit-learn StandardScaler	Preprocessing	Scale station features before PCA/K-Means and numeric ML features when needed.
Scikit-learn PCA	Dimensionality reduction	Reduce station feature space and visualise structure.
Scikit-learn KMeans	Clustering	Station grouping based on numerical network profiles.
Scikit-learn LogisticRegression	Baseline classifier	Link prediction.
Scikit-learn RandomForestClassifier	Non-linear classifier	Link prediction comparison.
Scikit-learn metrics	Model evaluation	Precision, recall, F1, ROC-AUC, PR-AUC, confusion matrix, silhouette score.
Matplotlib / Plotly	Visualisation	Static report plots / interactive

Tool / library	Use it here	Purpose
		dashboard charts.
Folium or Plotly map	Interactive mapping	Station/rail-line visualisation in the application.
SQLite	Optional structured storage	Processed station, edge, feature and model-result tables; useful if CSVs become messy.
Joblib	Model persistence	Save fitted scaler/PCA/ML models for the Streamlit app.
Streamlit	Front-end application	Dashboard, filters, maps, station search and model-result pages.
Jupyter Notebook	Exploration	EDA and experimentation; move stable logic into .py modules later.
VS Code	Project development	Organise scripts, Streamlit app and reusable functions.

Deep learning, LSTM, GNNs, Spark, Hadoop or cloud deployment are not required for this version. Add them only if the basic graph and ML pipeline works reliably and the team can explain why the additional technology is necessary.

10. Implementation Plan and Folder Structure

Keep exploration notebooks separate from stable project code. A simple folder structure is enough:

```
railnet-india/
│
├── data/
│   ├── raw/
│   ├── interim/
│   └── processed/
├── notebooks/
│   ├── 01_data_understanding.ipynb
│   ├── 02_historical_eda.ipynb
│   ├── 03_gis_processing.ipynb
```

```
│   ├── 04_graph_analysis.ipynb
│   └── 05_ml_experiments.ipynb
├── src/
│   ├── data_cleaning.py
│   ├── gis_processing.py
│   ├── graph_builder.py
│   ├── graph_features.py
│   ├── clustering.py
│   └── link_prediction.py
├── models/
│   ├── scaler.joblib
│   ├── pca.joblib
│   └── link_model.joblib
├── outputs/
│   ├── figures/
│   ├── tables/
│   └── model_metrics/
├── app.py
├── requirements.txt
└── README.md
```

Recommended implementation order

1. Make the gauge-history notebook run correctly and produce clean plots.

2. Clean the station table and map the stations.

3. Build a small graph on one state/zone first; verify edges manually.

4. Scale up to the chosen study area once the graph logic is correct.

5. Calculate graph metrics and save station_features.csv.

6. Run PCA and K-Means; evaluate and interpret clusters.

7. Create a leakage-safe link-prediction train/test dataset.

8. Train Logistic Regression and Random Forest; save metrics and model.

9. Only after the analyses are stable, build the Streamlit interface.

10. Freeze the final dataset versions used in the report so results remain reproducible.

11. Model Evaluation and Validation

Avoid reporting only accuracy, particularly for link prediction where positive and negative examples may be imbalanced.

Component	Recommended validation	Reason
Historical analysis	Internal consistency checks, totals, missing values,	Shows that input data is

	source/vintage verification	trustworthy.
Clustering	Silhouette score, cluster sizes, PCA plot, original-feature averages per cluster	Checks whether clusters are separated and interpretable.
Link prediction	Precision, Recall, F1, ROC-AUC, PR-AUC, confusion matrix	Measures classification quality from several perspectives.
Ranking use-case	Precision@K (optional)	Useful if the model outputs a ranked list of candidate links.
Robustness experiment	Largest connected component ratio / efficiency after node removal	Quantifies structural degradation.

Avoiding data leakage in link prediction

This is one point worth understanding well for the viva. If an edge is held out as a test example but you calculate common-neighbour or shortest-path features using the full graph that still contains that edge, the model indirectly sees information from the answer. The safer approach is to remove test edges from the graph before calculating leakage-sensitive training/test features.

12. Dashboard / Application Design

The interface should be simple enough to demonstrate in a few minutes. Avoid turning the Streamlit app into a large web-development project.

Page	What it should show
Home / Overview	Project summary, headline network statistics, data-vintage warning and key plots.
Historical Trends	Gauge route km, gauge share, electrification and selected infrastructure comparisons.
Railway Map	Station and rail-line map with state/zone filters and optional gauge/electrification display where the source supports it.
Station Intelligence	Search a station and display its node attributes, degree, betweenness, closeness, PageRank and cluster.
Graph Analysis	Top central stations, community summary and vulnerability curves.
ML Results	PCA plot, K-Means clusters, silhouette score, link-prediction model

Page	What it should show
	comparison and confusion matrix.
Link Prediction Demo	Select two stations and display the model score as an experimental graph-based link likelihood, with a clear disclaimer.

13. Expected Results and Project Deliverables

Do not write numerical model results before running the final experiments. The project proposal should describe the expected form of results, while the final report should insert the actual measured values.

- A cleaned and documented railway data package.
- Historical gauge and infrastructure trend visualisations.
- A processed station/rail-line GIS layer for the chosen scope.
- A NetworkX graph with node/edge attributes.
- Centrality rankings and connected-component statistics.
- A network robustness comparison under targeted and random node removal.
- A station_features dataset.
- PCA visualisation and K-Means clustering results with interpretation.
- A link-prediction dataset, trained models and evaluation metrics.
- A Streamlit application integrating the major outputs.
- Final report, presentation and reproducible source code.

Minimum viable final submission

If time becomes limited, finish these first

Historical analytics + clean station dataset + railway graph + centrality analysis + PCA/K-Means + one well-evaluated link-prediction model + a simple Streamlit dashboard. The vulnerability experiment and advanced mapping can be treated as enhancements.

14. Semester Execution Plan

Period	Work
Weeks 1–2	Problem definition, source collection, data dictionary, raw-data archive.
Weeks 3–4	Cleaning gauge/station data; basic historical EDA.
Weeks 5–6	GIS processing and first station/rail-line map.
Weeks 7–8	Graph construction, validation and centrality analysis.

Period	Work
Week 9	Community / robustness experiment and station feature engineering.
Weeks 10–11	PCA, K-Means, cluster evaluation and interpretation.
Weeks 12–13	Link-prediction dataset, Logistic Regression, Random Forest, evaluation.
Week 14	Streamlit integration.
Week 15	Testing, result freeze, screenshots/plots, final report.
Week 16	Presentation, demo rehearsal and viva preparation.

15. Risks, Limitations and What Not to Claim

Risk / limitation	Why it matters	How to handle it
Incomplete OSM attributes	Gauge/electrification may be missing on some mapped lines.	Keep unknown as unknown; report completeness and use only reliable attributes.
Station/line matching errors	A station point may not match the intended rail segment automatically.	Validate a sample manually and document matching rules.
Very large graph	National processing may be slow.	Prototype on one state/zone; cache processed graph and features.
Historical source mismatch	Different publications may use different dates/definitions.	Carry data vintage and unit; do not merge incompatible rows.
Class imbalance	Link prediction may have far more non-edges than edges.	Use controlled negative sampling and report suitable metrics.
Data leakage	Test-edge information can enter graph features.	Remove held-out edges before leakage-sensitive feature calculation.
Overclaiming	Model score may be mistaken for a real planning	Describe it as an experimental graph-based link prediction

	recommendation.	result only.

16. Suggested Final Report Chapter Structure

Chapter	Include
Chapter 1 – Introduction	Background, motivation, problem statement, aim, objectives, scope and contribution.
Chapter 2 – Literature Review	Railway/network analytics, GIS in transport, centrality, clustering and graph link prediction. Use recent scholarly work, not generic web articles.
Chapter 3 – Data and System Design	Data sources, variables, quality issues, architecture, database/processed schema and technology stack.
Chapter 4 – Methodology	Cleaning, historical EDA, GIS processing, graph construction, graph analytics, PCA/K-Means, link prediction and evaluation design.
Chapter 5 – Implementation	Python modules, important functions, dashboard pages and deployment/run procedure.
Chapter 6 – Results and Discussion	Actual plots, centrality results, vulnerability curves, clustering, ML metrics and interpretation.
Chapter 7 – Conclusion and Future Work	Main findings, limitations and realistic extensions such as temporal graphs or GNNs.

The contribution of the project is the integration of heterogeneous railway statistics and geospatial network data into a reproducible analytics pipeline that supports temporal analysis, graph-based station importance, unsupervised station profiling, supervised graph link prediction and interactive visual exploration.

Source	Link	Use / note
Indian Railways Year Book / statistical publications	https://indianrailways.gov.in/railwayboard/view_section_new.jsp?id=0,1,304,366,554&lang=0	Official railway statistics and yearbooks.
Open Government Data Platform India – Railway Station catalog	https://www.data.gov.in/catalog/railway-station	Government station-data

		catalog; check the resource date before use.
Open Government Data Platform India – Indian Railways Train Time Table	https://www.data.gov.in/catalog/indian-railways-train-time-table	Optional timetable/route data; verify vintage.
OpenStreetMap / Geofabrik India extract	https://download.geofabrik.de/asia/india.html	Rail-line and station GIS data. Follow OpenStreetMap attribution/licensing requirements.
OpenStreetMap railway tagging documentation	https://wiki.openstreetmap.org/wiki/Tag:railway%3Drail	Explains common railway tags; actual completeness varies.
DataMeet Railways repository	https://github.com/datameet/railways	Community railway/station GIS files; validate important fields.